

**UNIVERSIDAD PERUANA LOS ANDES  
FACULTAD DE INGENIERÍA  
ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS Y  
COMPUTACIÓN**



**ALGORITMOS Y ESTRUCTURA DE DATOS**

**ACTIVIDAD 1: IMPLEMENTACIÓN DE OPERACIONES  
CON ARREGLOS EN JAVA**

**ALUMNO:**

**> VEGA MUÑOZ JHONSTYN YERAY**

**DOCENTE:**

**> FERNANDEZ BEJARANO RAUL ENRIQUE**

**CICLO:**

**IV**

**SECCIÓN:**

**A1**

**HUANCAYO - PERÚ  
2026**

## **1. INTRODUCCIÓN**

La presente actividad desarrolla una aplicación en Java utilizando Apache NetBeans para trabajar con arreglos de números enteros. La solución se organiza en dos clases: Arreglo, que concentra los datos y las operaciones sobre el arreglo, y AplicaArreglo, que contiene el método main () y un menú desde el cual el usuario ejecuta cada operación. Esta separación permite mantener la lógica del arreglo independiente de la interacción con el usuario y facilita la lectura, prueba y mantenimiento del programa.

La actividad comprende la eliminación, modificación e inserción de elementos; el cálculo de sumas, menor, mayor y promedio; y el desplazamiento de elementos hacia la última posición. Para evidenciar el desarrollo se incluyen el enunciado, el TDA trabajado en clase, la estructura del proyecto, las capturas del código y las pruebas de ejecución realizadas.

## **2. ENUNCIADO DE LA ACTIVIDAD**

La Actividad 1 solicita implementar ocho métodos sobre un arreglo creado previamente. Cada método resuelve una operación específica y luego es llamado desde el menú principal de la clase AplicaArreglo.

### **ACTIVIDAD 1**

1. Elaborar un método para eliminar un elemento "x", del arreglo creado anteriormente.
2. Elaborar un método para modificar un elemento "x".
3. Elaborar un método para insertar un elemento "x" en una posición "y"
4. Elaborar un método para sumar los elementos del arreglo.
5. Elaborar un método para hallar el número menor, el número mayor y el promedio de los elementos del arreglo.
6. Elaborar un método para suma de Impares los elementos del arreglo.
7. Elaborar un método para mover el Primer elemento a la Última posición del arreglo.
8. Elaborar un método para mover el Primer elemento Par al Final del arreglo.

Los ocho requerimientos son: eliminar un elemento x; modificar un elemento x; insertar un elemento x en una posición y; sumar todos los elementos; hallar el menor, el mayor y el promedio; sumar los elementos impares; mover el primer elemento a la última posición; y mover el primer elemento par al final del arreglo.

## **3. OBJETIVOS**

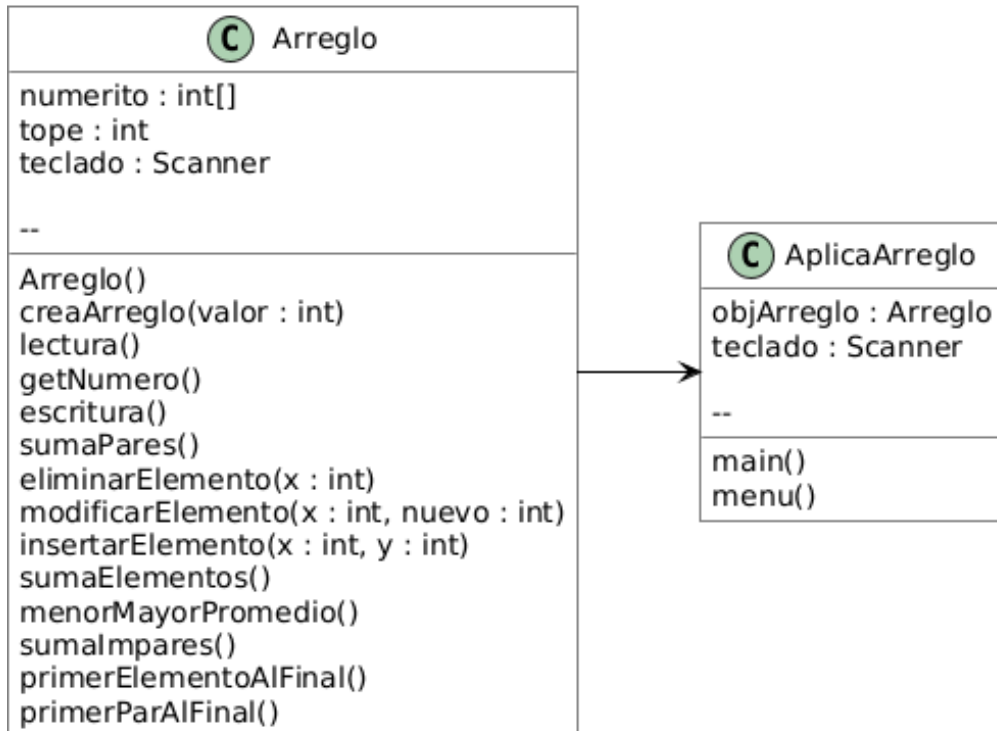
Objetivo general. Implementar operaciones sobre arreglos de enteros en Java mediante métodos independientes, utilizando estructuras condicionales, ciclos y una organización básica orientada a objetos.

Objetivos específicos. Aplicar recorridos lineales para localizar elementos; modificar la estructura lógica del arreglo mediante desplazamientos; efectuar cálculos acumulativos y estadísticos; y comprobar cada operación mediante un menú interactivo y pruebas de ejecución.

## **TDA Y ORGANIZACIÓN DE CLASES**

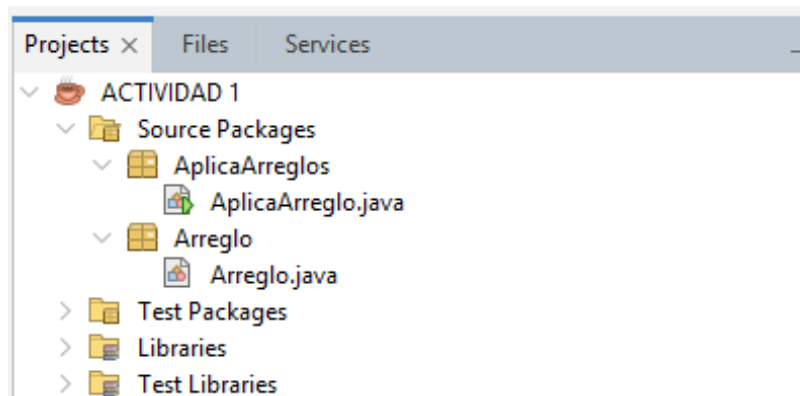
El TDA fue planteado en clase con dos clases relacionadas. La clase Arreglo administra el arreglo numerito, la variable tope y los métodos que operan sobre los datos. La clase AplicaArreglo crea un objeto de Arreglo y presenta el menú que permite llamar a dichos métodos.

## TDA - ACTIVIDAD 1



## 4. ESTRUCTURA DEL PROYECTO EN APACHE NETBEANS

El proyecto se organiza en dos paquetes separados. *AplicaArreglos* contiene la clase *AplicaArreglo.java*, responsable de la ejecución y del menú. *Arreglo* contiene la clase *Arreglo.java*, donde se encuentran el arreglo y todos los métodos de procesamiento. Esta separación permite que la clase principal utilice la lógica de *Arreglo* mediante el objeto *objArreglo*.



## 5. DESARROLLO Y EXPLICACIÓN DEL CÓDIGO

### 5.1. Clase *AplicaArreglo*: control de la ejecución

La clase *AplicaArreglo* funciona como punto de entrada del programa. Importa la clase *Arreglo* y *Scanner*, crea el objeto *objArreglo* y ejecuta *menu()* desde *main()*. El ciclo *do-while* mantiene el programa activo hasta que el usuario selecciona la opción 0. Las variables *x*, *y* y *nuevo* se utilizan para recibir los datos requeridos por las operaciones de eliminar, insertar y modificar.

```
AplicaArreglo.java x
Source History
1 package AplicaArreglos;
2
3 import Arreglo.Arreglo;
4 import java.util.Scanner;
5
6 public class AplicaArreglo {
7
8     static Arreglo objArreglo = new Arreglo();
9     static Scanner teclado = new Scanner(System.in);
10
11     public static void main(String[] args) {
12         menu();
13     }
14
15     static void menu() {
16         int opcion; int tam; int x; int y; int nuevo;
17
18         do {
19
20             System.out.println("");
21             System.out.println("MENU DE OPCIONES");
22             System.out.println("-----");
23             System.out.println("1.- Crear el Arreglo");
24             System.out.println("2.- Ingresar Datos");
25             System.out.println("3.- Mostrar Datos");
26             System.out.println("4.- Suma de Pares");
27             System.out.println("5.- Eliminar un Elemento");
28             System.out.println("6.- Modificar un Elemento");
29             System.out.println("7.- Insertar un Elemento");
30             System.out.println("8.- Sumar todos los Elementos");
31             System.out.println("9.- Menor, Mayor y Promedio");
32             System.out.println("10.- Suma de Impares");
33             System.out.println("11.- Primer Elemento al Final");
34             System.out.println("12.- Primer Elemento Par al Final");
35             System.out.println("-----");
36             System.out.println("0.- Salir");
37             System.out.println("-----");
38
39             System.out.print("Ingrese una alternativa: ");
40             opcion = teclado.nextInt();
41
42             System.out.println("");

```

El menú enumera todas las operaciones disponibles. Se utiliza una variable `opcion` leída con `Scanner` para decidir qué bloque `if` debe ejecutarse. Esta forma es sencilla para una práctica inicial porque cada opción queda asociada directamente con un método de la clase `Arreglo`.

```

//OPCION 1 - CREAR ARREGLO
if (opcion == 1) {

    System.out.print("Ingrese tamaño del Arreglo: ");
    tam = teclado.nextInt();

    objArreglo.creaArreglo(tam);

    System.out.println("Arreglo creado correctamente");
}

// OPCION 2 - INGRESAR DATOS
if (opcion == 2) {

    objArreglo.lectura();
}

// OPCION 3 - MOSTRAR DATOS
if (opcion == 3) {

    objArreglo.escritura();
}

// OPCION 4 - SUMA DE PARES
if (opcion == 4) {

    objArreglo.sumaPares();
}

```

La opción 1 solicita el tamaño y llama a creaArreglo(tam), con lo que se reserva el espacio inicial. La opción 2 llama a lectura() para ingresar un dato por vez; la opción 3 llama a escritura() para visualizar los valores almacenados; y la opción 4 llama a sumaPares() para recorrer el arreglo y sumar únicamente los números divisibles entre dos. Estas opciones constituyen la base sobre la que se prueban los ocho nuevos requerimientos.

## 5.2. Punto 1: eliminar un elemento x

```

// ACTIVIDAD 1 - PUNTO 1
// OPCION 5 - ELIMINAR ELEMENTO

if (opcion == 5) {

    System.out.print("Ingrese elemento a eliminar: ");
    x = teclado.nextInt();

    objArreglo.eliminarElemento(x);
}

```

En AplicaArreglo se solicita el valor que se desea eliminar y se envía como argumento x. Se mantiene la lectura del dato en la clase principal porque esta clase es la encargada de la interacción con el usuario, mientras que la modificación del arreglo se realiza en la clase Arreglo.

```

71
72 // Sumar numeros pares
73 public void sumaPares() {
74
75     int suma = 0;
76
77     for (int i = 0; i < tope; i++) {
78
79         if (numerito[i] % 2 == 0) {
80
81             suma = suma + numerito[i];
82         }
83     }
84
85     System.out.println("La suma de Pares es: " + suma);
86
87 }
88
89 // ACTIVIDAD 1 - PUNTO 1
90 // Eliminar un elemento "x"
91
92 public void eliminarElemento(int x) {
93
94     for (int i = 0; i < tope; i++) {
95
96         if (numerito[i] == x) {
97
98             // Mover elementos hacia la izquierda
99             for (int j = i; j < tope - 1; j++) {
100                 numerito[j] = numerito[j + 1];
101             }
102
103             tope--;
104
105             System.out.println("Elemento eliminado");
106             return;
107         }
108     }
109
110     System.out.println("Elemento no encontrado");
111 }

```

El método eliminarElemento(x) recorre las posiciones hasta encontrar la primera coincidencia. Cuando la encuentra, un segundo for desplaza hacia la izquierda todos los elementos posteriores; de esta manera se cubre el espacio del valor eliminado. Luego tope-- reduce en una unidad la cantidad lógica de elementos y return finaliza el método para evitar continuar buscando después de la eliminación.

### 5.3. Punto 2: modificar un elemento x

```

// ACTIVIDAD 1 - PUNTO 2
// OPCION 6 - MODIFICAR ELEMENTO

if (opcion == 6) {

    System.out.print("Ingrese elemento a modificar: ");
    x = teclado.nextInt();

    System.out.print("Ingrese nuevo valor: ");
    nuevo = teclado.nextInt();

    objArreglo.modificarElemento(x, nuevo);
}

```

Para modificar un dato se necesitan dos valores: x representa el elemento buscado y nuevo

representa el valor que lo reemplazará. Ambos se capturan con Scanner y se envían juntos al método modificarElemento(x, nuevo).

```
112 // ACTIVIDAD 1 - PUNTO 2
113 // Modificar un elemento "x"
114
115 public void modificarElemento(int x, int nuevo) {
116
117     for (int i = 0; i < tope; i++) {
118
119         if (numerito[i] == x) {
120
121             numerito[i] = nuevo;
122
123             System.out.println("Elemento modificado");
124             return;
125         }
126     }
127
128     System.out.println("Elemento no encontrado");
129 }
```

El método realiza una búsqueda lineal mediante un for. Al encontrar numerito[i] == x, sustituye directamente esa posición por nuevo. El uso de return hace que se modifique únicamente la primera coincidencia, conservando una lógica simple y predecible para la actividad. Si el recorrido termina sin coincidencias, se informa que el elemento no fue encontrado.

#### 5.4. Punto 3: insertar un elemento x en una posición y

```
// ACTIVIDAD 1 - PUNTO 3
// OPCION 7 - INSERTAR ELEMENTO

if (opcion == 7) {

    System.out.print("Ingrese elemento a insertar: ");
    x = teclado.nextInt();

    System.out.print("Ingrese posicion: ");
    y = teclado.nextInt();

    objArreglo.insertarElemento(x, y);
}
```

La opción 7 solicita el nuevo elemento y la posición donde se desea insertar. El valor x contiene el número y la variable y representa una posición lógica iniciada en 1 para facilitar su uso desde el menú.

```

131 // ACTIVIDAD 1 - PUNTO 3
132 // Insertar un elemento "x"
133 // en una posicion "y"
134
135 public void insertarElemento(int x, int y) {
136
137     if (y < 1 || y > tope + 1) {
138         System.out.println("Posicion incorrecta");
139         return;
140     }
141
142     // Crear un nuevo arreglo con un espacio mas
143     int[] nuevoArreglo = new int[numerito.length + 1];
144
145     // Copiar los elementos anteriores
146     for (int i = 0; i < y - 1; i++) {
147         nuevoArreglo[i] = numerito[i];
148     }
149
150     // Insertar el nuevo elemento
151     nuevoArreglo[y - 1] = x;
152
153     // Copiar los elementos restantes
154     for (int i = y - 1; i < tope; i++) {
155         nuevoArreglo[i + 1] = numerito[i];
156     }
157
158     numerito = nuevoArreglo;
159     tope++;
160
161     System.out.println("Elemento insertado");
162 }
163

```

Primero se valida que la posición se encuentre entre 1 y tope + 1. Debido a que los arreglos de Java tienen tamaño fijo, se crea nuevoArreglo con una posición adicional. Luego se copian los elementos anteriores a la posición, se coloca x en y - 1 y se copian los elementos restantes una posición más adelante. Finalmente numerito pasa a referenciar el nuevo arreglo y tope++ registra que existe un elemento adicional.

### 5.5. Punto 4: sumar todos los elementos

```

// ACTIVIDAD 1 - PUNTO 4
// OPCION 8 - SUMAR ELEMENTOS

if (opcion == 8) {

    objArreglo.sumaElementos();
}

```

La opción no necesita pedir datos adicionales porque el método trabaja con los elementos que ya se encuentran almacenados. Por ello únicamente se llama a objArreglo.sumaElementos().

```

164      // ACTIVIDAD 1 - PUNTO 4
165      // Sumar todos los elementos del arreglo
166
167      public void sumaElementos() {
168
169          int suma = 0;
170
171          for (int i = 0; i < tope; i++) {
172              suma = suma + numerito[i];
173          }
174
175          System.out.println("La suma de los elementos es: " + suma);
176      }
177

```

Se inicializa suma en cero y se recorre el arreglo desde la posición 0 hasta tope - 1. En cada iteración se acumula numerito[i]. Esta estructura es suficiente porque la suma requiere visitar una sola vez cada elemento válido del arreglo.

### 5.6. Punto 5: número menor, número mayor y promedio

```

      // ACTIVIDAD 1 - PUNTO 5
      // OPCION 9 - MENOR, MAYOR Y PROMEDIO

      if (opcion == 9) {

          objArreglo.menorMayorPromedio();
      }

```

El cálculo se concentra completamente en la clase Arreglo, por lo que desde el menú solo se realiza la llamada al método. Esto evita duplicar lógica en la clase principal.

```

178 // ACTIVIDAD 1 - PUNTO 5
179 // Hallar numero menor, mayor y promedio
180
181 public void menorMayorPromedio() {
182
183     if (tope == 0) {
184         System.out.println("No existen elementos");
185         return;
186     }
187     int menor = numerito[0];
188     int mayor = numerito[0];
189     int suma = 0;
190
191     for (int i = 0; i < tope; i++) {
192
193         if (numerito[i] < menor) {
194             menor = numerito[i];
195         }
196
197         if (numerito[i] > mayor) {
198             mayor = numerito[i];
199         }
200
201         suma = suma + numerito[i];
202     }
203
204     double promedio = (double) suma / tope;
205
206     System.out.println("Numero menor: " + menor);
207     System.out.println("Numero mayor: " + mayor);
208     System.out.println("Promedio: " + promedio);
209
210 }
211

```

Antes de calcular se valida que existan elementos. menor y mayor se inicializan con numerito[0], porque un dato real del arreglo es un punto de partida válido. Durante un único recorrido se actualiza menor cuando aparece un valor inferior, mayor cuando aparece un valor superior y, al mismo tiempo, se acumula suma. Finalmente se convierte la suma a double antes de dividir entre tope para evitar una división entera y conservar los decimales del promedio.

### 5.7. Punto 6: suma de elementos impares

```

// ACTIVIDAD 1 - PUNTO 6
// OPCION 10 - SUMA DE IMPARES

if (opcion == 10) {

    objArreglo.sumaImpares();
}

```

La suma de impares no requiere información adicional del usuario, porque el método analiza directamente los elementos ya registrados.

```

212 // ACTIVIDAD 1 - PUNTO 6
213 // Sumar los elementos impares
214
215 public void sumaImpares() {
216
217     int suma = 0;
218
219     for (int i = 0; i < tope; i++) {
220
221         if (numerito[i] % 2 != 0) {
222             suma = suma + numerito[i];
223         }
224     }
225
226     System.out.println("La suma de Impares es: " + suma);
227 }
228

```

La condición `numerito[i] % 2 != 0` comprueba que el residuo de dividir el número entre dos sea distinto de cero, característica de un número impar. Solo los valores que cumplen la condición se agregan a suma, y al finalizar se imprime el acumulado.

### 5.8. Punto 7: mover el primer elemento a la última posición

```

// ACTIVIDAD 1 - PUNTO 7
// OPCION 11 - PRIMER ELEMENTO AL FINAL

if (opcion == 11) {

    objArreglo.primerElementoAlFinal();

}

```

Desde el menú se ejecuta directamente el método porque la operación no necesita parámetros: siempre trabaja con el elemento ubicado en la primera posición.

```

18
19 // ACTIVIDAD 1 - PUNTO 7
20 // Mover el primer elemento
21 // a la ultima posicion
22
23 public void primerElementoAlFinal() {
24
25     if (tope == 0) {
26         System.out.println("No existen elementos");
27         return;
28     }
29
30     int aux = numerito[0];
31
32     // Mover los elementos hacia la izquierda
33     for (int i = 0; i < tope - 1; i++) {
34         numerito[i] = numerito[i + 1];
35     }
36
37     numerito[tope - 1] = aux;
38     System.out.println("Primer elemento movido al final");
39 }
40

```

Primero se comprueba que el arreglo tenga elementos. El valor `numerito[0]` se guarda temporalmente en `aux` para no perderlo. Después un `for` desplaza cada posición una unidad hacia la izquierda y, cuando termina el recorrido, `aux` se coloca en `numerito[tope - 1]`. El resultado equivale a rotar una posición el contenido del arreglo hacia la izquierda.

### 5.9. Punto 8: mover el primer elemento par al final

```
// ACTIVIDAD 1 - PUNTO 8
// OPCION 12 - PRIMER PAR AL FINAL

] if (opcion == 12) {
    objArreglo.primerParAlFinal();
- }
```

La operación busca por sí misma cuál es el primer número par, por lo que tampoco requiere que el usuario ingrese un valor adicional desde el menú.

```
250
251 // ACTIVIDAD 1 - PUNTO 8
252 // Mover el primer elemento par
253 // al final del arreglo
254
255 public void primerParAlFinal() {
256
257     for (int i = 0; i < tope; i++) {
258
259         // Encontrar el primer numero par
260         if (numerito[i] % 2 == 0) {
261
262             int aux = numerito[i];
263
264             // Mover los elementos hacia la izquierda
265             for (int j = i; j < tope - 1; j++) {
266                 numerito[j] = numerito[j + 1];
267             }
268
269             numerito[tope - 1] = aux;
270
271             System.out.println("Primer elemento par movido al final");
272             return;
273         }
274     }
275
276     System.out.println("No existen numeros pares");
277 }
278 }
```

El primer `for` localiza la primera posición cuyo valor cumple `numerito[i] % 2 == 0`. Ese número se almacena en `aux` y, desde su posición, los siguientes elementos se desplazan una posición hacia la izquierda. Finalmente el número par se coloca al final. El `return` es importante porque detiene la búsqueda después de mover el primer par; si no existe ninguno, el método muestra el mensaje correspondiente.

### 5.10. Salida del programa

```
// OPCION 0 - SALIR

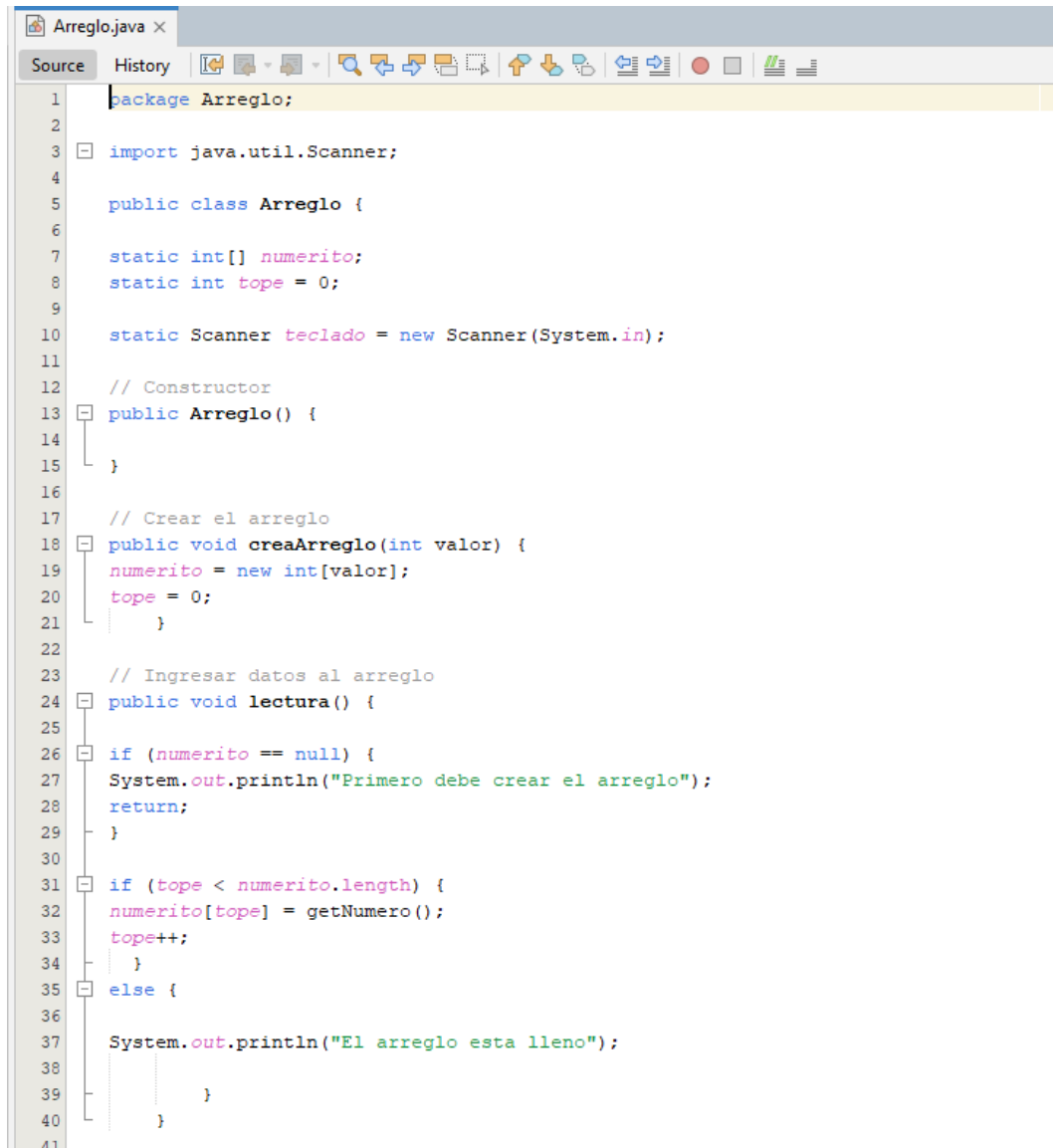
    if (opcion == 0) {

        System.out.println("SALIENDO DEL PROGRAMA");
    }

    } while (opcion != 0);
}
```

La opción 0 imprime el mensaje “SALIENDO DEL PROGRAMA”. Después, la condición while (opcion != 0) deja de cumplirse y el ciclo termina. Esto proporciona un cierre controlado y evita finalizar el programa de manera abrupta.

### 5.11. Métodos base de la clase Arreglo



```
Arreglo.java x
Source History
1 package Arreglo;
2
3 import java.util.Scanner;
4
5 public class Arreglo {
6
7     static int[] numerito;
8     static int tope = 0;
9
10    static Scanner teclado = new Scanner(System.in);
11
12    // Constructor
13    public Arreglo() {
14    }
15
16    // Crear el arreglo
17    public void creaArreglo(int valor) {
18        numerito = new int[valor];
19        tope = 0;
20    }
21
22    // Ingresar datos al arreglo
23    public void lectura() {
24
25        if (numerito == null) {
26            System.out.println("Primero debe crear el arreglo");
27            return;
28        }
29
30        if (tope < numerito.length) {
31            numerito[tope] = getNumero();
32            tope++;
33        }
34        else {
35
36            System.out.println("El arreglo esta lleno");
37
38        }
39    }
40 }
41
```

numerito almacena los números y tope indica cuántas posiciones contienen datos válidos. creaArreglo(valor) reserva el arreglo con el tamaño elegido y reinicia tope. lectura() valida que el arreglo exista y que aún tenga espacio antes de almacenar un nuevo número; de esta forma se evita acceder a un arreglo no creado o superar su capacidad.

```

42 // Pedir un numero
43 public int getNumero() {
44
45     int valor;
46
47     System.out.print("Ingrese un Numero: ");
48     valor = teclado.nextInt();
49     return valor;
50 }
51
52 // Mostrar los elementos
53
54 public void escritura() {
55
56     System.out.println("\tLISTADO");
57     System.out.println("\t-----");
58
59     if (tope > 0) {
60
61         for (int i = 0; i < tope; i++) {
62             System.out.print(numerito[i] + "\t");
63         }
64
65     } else {
66         System.out.println("No existen elementos");
67     }
68
69     System.out.println("\n");
70 }
71

```

getNumero() concentra la lectura de un entero y devuelve el valor al método que lo necesita. escritura() recorre únicamente desde 0 hasta tope - 1 y muestra los elementos almacenados. Si tope es cero, informa que no existen elementos. Separar estas tareas en métodos hace que el código sea más ordenado y reutilizable.

## 6. PRUEBAS DE EJECUCIÓN Y ANÁLISIS DE RESULTADOS

Para comprobar el funcionamiento se creó un arreglo de tamaño 5 y se ingresaron inicialmente los valores 10, 5, 8, 3 y 20. A partir de estos datos se ejecutaron, en orden, las opciones del menú correspondientes a los ocho puntos de la actividad.

```
Output - ACTIVIDAD 1 (run) x
run:
MENU DE OPCIONES
-----
1.- Crear el Arreglo
2.- Ingresar Datos
3.- Mostrar Datos
4.- Suma de Pares
5.- Eliminar un Elemento
6.- Modificar un Elemento
7.- Insertar un Elemento
8.- Sumar todos los Elementos
9.- Menor, Mayor y Promedio
10.- Suma de Impares
11.- Primer Elemento al Final
12.- Primer Elemento Par al Final
-----
0.- Salir
-----
Ingrese una alternativa: 1

Ingrese tamano del Arreglo: 5
Arreglo creado correctamente

MENU DE OPCIONES
-----
1.- Crear el Arreglo
2.- Ingresar Datos
3.- Mostrar Datos
4.- Suma de Pares
5.- Eliminar un Elemento
6.- Modificar un Elemento
7.- Insertar un Elemento
8.- Sumar todos los Elementos
9.- Menor, Mayor y Promedio
10.- Suma de Impares
11.- Primer Elemento al Final
12.- Primer Elemento Par al Final
-----
0.- Salir
-----
Ingrese una alternativa: 2

Ingrese un Numero: 10
```

La opción 1 crea un arreglo con capacidad para cinco elementos. A continuación se utiliza la opción 2 para registrar el primer dato, 10. Esta prueba confirma que la creación y la primera lectura funcionan correctamente.

```

MENU DE OPCIONES
-----
1.- Crear el Arreglo
2.- Ingresar Datos
3.- Mostrar Datos
4.- Suma de Pares
5.- Eliminar un Elemento
6.- Modificar un Elemento
7.- Insertar un Elemento
8.- Sumar todos los Elementos
9.- Menor, Mayor y Promedio
10.- Suma de Impares
11.- Primer Elemento al Final
12.- Primer Elemento Par al Final
-----
0.- Salir
-----

Ingrese una alternativa: 2

Ingrese un Numero: 5

MENU DE OPCIONES
-----
1.- Crear el Arreglo
2.- Ingresar Datos
3.- Mostrar Datos
4.- Suma de Pares
5.- Eliminar un Elemento
6.- Modificar un Elemento
7.- Insertar un Elemento
8.- Sumar todos los Elementos
9.- Menor, Mayor y Promedio
10.- Suma de Impares
11.- Primer Elemento al Final
12.- Primer Elemento Par al Final
-----
0.- Salir
-----

Ingrese una alternativa: 2

Ingrese un Numero: 8

MENU DE OPCIONES
-----
1.- Crear el Arreglo
2.- Ingresar Datos
3.- Mostrar Datos
4.- Suma de Pares
5.- Eliminar un Elemento

```

La opción 2 se ejecuta nuevamente para añadir los siguientes datos. El programa regresa al menú después de cada ingreso, lo que permite continuar registrando valores de manera controlada.

```
Output - ACTIVIDAD 1 (run) x
5.- Eliminar un Elemento
6.- Modificar un Elemento
7.- Insertar un Elemento
8.- Sumar todos los Elementos
9.- Menor, Mayor y Promedio
10.- Suma de Impares
11.- Primer Elemento al Final
12.- Primer Elemento Par al Final
-----
0.- Salir
-----
Ingrese una alternativa: 2

Ingrese un Numero: 3

MENU DE OPCIONES
-----
1.- Crear el Arreglo
2.- Ingresar Datos
3.- Mostrar Datos
4.- Suma de Pares
5.- Eliminar un Elemento
6.- Modificar un Elemento
7.- Insertar un Elemento
8.- Sumar todos los Elementos
9.- Menor, Mayor y Promedio
10.- Suma de Impares
11.- Primer Elemento al Final
12.- Primer Elemento Par al Final
-----
0.- Salir
-----
Ingrese una alternativa: 2

Ingrese un Numero: 20

MENU DE OPCIONES
-----
1.- Crear el Arreglo
2.- Ingresar Datos
3.- Mostrar Datos
4.- Suma de Pares
5.- Eliminar un Elemento
6.- Modificar un Elemento
7.- Insertar un Elemento
8.- Sumar todos los Elementos
9.- Menor, Mayor y Promedio
10.- Suma de Impares
11.- Primer Elemento al Final
```

Con estas entradas se completa el conjunto inicial de cinco elementos: 10, 5, 8, 3 y 20. El arreglo queda preparado para realizar las operaciones solicitadas.

```
Output - ACTIVIDAD 1 (run) x
11.- Primer Elemento al Final
12.- Primer Elemento Par al Final
-----
0.- Salir
-----
Ingrese una alternativa: 3

      LISTADO
      -----
10      5      8      3      20

MENU DE OPCIONES
-----
1.- Crear el Arreglo
2.- Ingresar Datos
3.- Mostrar Datos
4.- Suma de Pares
5.- Eliminar un Elemento
6.- Modificar un Elemento
7.- Insertar un Elemento
8.- Sumar todos los Elementos
9.- Menor, Mayor y Promedio
10.- Suma de Impares
11.- Primer Elemento al Final
12.- Primer Elemento Par al Final
-----
0.- Salir
-----
Ingrese una alternativa: 4

La suma de Pares es: 38

MENU DE OPCIONES
-----
1.- Crear el Arreglo
2.- Ingresar Datos
3.- Mostrar Datos
4.- Suma de Pares
5.- Eliminar un Elemento
6.- Modificar un Elemento
7.- Insertar un Elemento
8.- Sumar todos los Elementos
9.- Menor, Mayor y Promedio
10.- Suma de Impares
11.- Primer Elemento al Final
12.- Primer Elemento Par al Final
-----
0.- Salir
```

La opción 3 muestra 10, 5, 8, 3 y 20. Luego la opción 4 obtiene 38 como suma de pares, resultado de 10 + 8 + 20. La salida confirma que el recorrido y la condición de paridad son correctos.

```
Output - ACTIVIDAD 1 (run) x
0.- Salir
-----
Ingrese una alternativa: 5
Ingrese elemento a eliminar: 8
Elemento eliminado

MENU DE OPCIONES
-----
1.- Crear el Arreglo
2.- Ingresar Datos
3.- Mostrar Datos
4.- Suma de Pares
5.- Eliminar un Elemento
6.- Modificar un Elemento
7.- Insertar un Elemento
8.- Sumar todos los Elementos
9.- Menor, Mayor y Promedio
10.- Suma de Impares
11.- Primer Elemento al Final
12.- Primer Elemento Par al Final
-----
0.- Salir
-----
Ingrese una alternativa: 3

LISTADO
-----
10      5      3      20

MENU DE OPCIONES
-----
1.- Crear el Arreglo
2.- Ingresar Datos
3.- Mostrar Datos
4.- Suma de Pares
5.- Eliminar un Elemento
6.- Modificar un Elemento
7.- Insertar un Elemento
8.- Sumar todos los Elementos
9.- Menor, Mayor y Promedio
10.- Suma de Impares
11.- Primer Elemento al Final
12.- Primer Elemento Par al Final
-----
0.- Salir
-----
Ingrese una alternativa: 6
```

Se selecciona la opción 5 y se ingresa 8. El mensaje confirma la eliminación y, al volver a mostrar el arreglo, se observa 10, 5, 3 y 20; los elementos posteriores se desplazaron correctamente hacia la izquierda.

```
Output - ACTIVIDAD 1 (run) x
Ingrese una alternativa: 6
Ingrese elemento a modificar: 5
Ingrese nuevo valor: 15
Elemento modificado

MENU DE OPCIONES
-----
1.- Crear el Arreglo
2.- Ingresar Datos
3.- Mostrar Datos
4.- Suma de Pares
5.- Eliminar un Elemento
6.- Modificar un Elemento
7.- Insertar un Elemento
8.- Sumar todos los Elementos
9.- Menor, Mayor y Promedio
10.- Suma de Impares
11.- Primer Elemento al Final
12.- Primer Elemento Par al Final
-----
0.- Salir
-----
Ingrese una alternativa: 3

LISTADO
-----
10    15    3    20

MENU DE OPCIONES
-----
1.- Crear el Arreglo
2.- Ingresar Datos
3.- Mostrar Datos
4.- Suma de Pares
5.- Eliminar un Elemento
6.- Modificar un Elemento
7.- Insertar un Elemento
8.- Sumar todos los Elementos
9.- Menor, Mayor y Promedio
10.- Suma de Impares
11.- Primer Elemento al Final
12.- Primer Elemento Par al Final
-----
0.- Salir
-----
Ingrese una alternativa: 7
```

La opción 6 busca el valor 5 y lo reemplaza por 15. La visualización posterior muestra 10, 15, 3 y 20, comprobando que se modificó la primera coincidencia sin alterar los demás elementos.

```
Output - ACTIVIDAD 1 (run) x
Ingrese elemento a insertar: 50
Ingrese posicion: 2
Elemento insertado

MENU DE OPCIONES
-----
1.- Crear el Arreglo
2.- Ingresar Datos
3.- Mostrar Datos
4.- Suma de Pares
5.- Eliminar un Elemento
6.- Modificar un Elemento
7.- Insertar un Elemento
8.- Sumar todos los Elementos
9.- Menor, Mayor y Promedio
10.- Suma de Impares
11.- Primer Elemento al Final
12.- Primer Elemento Par al Final
-----
0.- Salir
-----
Ingrese una alternativa: 3

LISTADO
-----
10    50    15    3    20

MENU DE OPCIONES
-----
1.- Crear el Arreglo
2.- Ingresar Datos
3.- Mostrar Datos
4.- Suma de Pares
5.- Eliminar un Elemento
6.- Modificar un Elemento
7.- Insertar un Elemento
8.- Sumar todos los Elementos
9.- Menor, Mayor y Promedio
10.- Suma de Impares
11.- Primer Elemento al Final
12.- Primer Elemento Par al Final
-----
0.- Salir
-----
Ingrese una alternativa: 8

La suma de los elementos es: 98
```

La opción 7 inserta el valor 50 en la posición 2. El arreglo pasa a ser 10, 50, 15, 3 y 20. Después, la opción 8 calcula una suma total de 98, que corresponde a la suma de los cinco valores mostrados.

```

MENU DE OPCIONES
-----
1.- Crear el Arreglo
2.- Ingresar Datos
3.- Mostrar Datos
4.- Suma de Pares
5.- Eliminar un Elemento
6.- Modificar un Elemento
7.- Insertar un Elemento
8.- Sumar todos los Elementos
9.- Menor, Mayor y Promedio
10.- Suma de Impares
11.- Primer Elemento al Final
12.- Primer Elemento Par al Final
-----
0.- Salir
-----
Ingrese una alternativa: 9

Numero menor: 3
Numero mayor: 50
Promedio: 19.6

MENU DE OPCIONES
-----
1.- Crear el Arreglo
2.- Ingresar Datos
3.- Mostrar Datos
4.- Suma de Pares
5.- Eliminar un Elemento
6.- Modificar un Elemento
7.- Insertar un Elemento
8.- Sumar todos los Elementos
9.- Menor, Mayor y Promedio
10.- Suma de Impares
11.- Primer Elemento al Final
12.- Primer Elemento Par al Final
-----
0.- Salir
-----
Ingrese una alternativa: 10

La suma de Impares es: 18

MENU DE OPCIONES
-----
1.- Crear el Arreglo

```

La opción 9 obtiene como menor 3, como mayor 50 y un promedio de 19.6. La opción 10 calcula 18 como suma de impares, resultado de  $15 + 3$ . Ambos resultados coinciden con los valores almacenados.

```
1.- Crear el Arreglo
2.- Ingresar Datos
3.- Mostrar Datos
4.- Suma de Pares
5.- Eliminar un Elemento
6.- Modificar un Elemento
7.- Insertar un Elemento
8.- Sumar todos los Elementos
9.- Menor, Mayor y Promedio
10.- Suma de Impares
11.- Primer Elemento al Final
12.- Primer Elemento Par al Final
-----
0.- Salir
-----
Ingrese una alternativa: 11

Primer elemento movido al final

MENU DE OPCIONES
-----
1.- Crear el Arreglo
2.- Ingresar Datos
3.- Mostrar Datos
4.- Suma de Pares
5.- Eliminar un Elemento
6.- Modificar un Elemento
7.- Insertar un Elemento
8.- Sumar todos los Elementos
9.- Menor, Mayor y Promedio
10.- Suma de Impares
11.- Primer Elemento al Final
12.- Primer Elemento Par al Final
-----
0.- Salir
-----
Ingrese una alternativa: 3

LISTADO
-----
50    15    3    20    10

MENU DE OPCIONES
-----
1.- Crear el Arreglo
2.- Ingresar Datos
3.- Mostrar Datos
4.- Suma de Pares
```

Antes de esta operación el arreglo es 10, 50, 15, 3, 20. La opción 11 mueve el primer elemento, 10, al final, dejando 50, 15, 3, 20, 10. La visualización confirma el desplazamiento realizado por el método.

```
Output - ACTIVIDAD 1 (run) x
4.- Suma de Pares
5.- Eliminar un Elemento
6.- Modificar un Elemento
7.- Insertar un Elemento
8.- Sumar todos los Elementos
9.- Menor, Mayor y Promedio
10.- Suma de Impares
11.- Primer Elemento al Final
12.- Primer Elemento Par al Final
-----
0.- Salir
-----
Ingrese una alternativa: 12

Primer elemento par movido al final

MENU DE OPCIONES
-----
1.- Crear el Arreglo
2.- Ingresar Datos
3.- Mostrar Datos
4.- Suma de Pares
5.- Eliminar un Elemento
6.- Modificar un Elemento
7.- Insertar un Elemento
8.- Sumar todos los Elementos
9.- Menor, Mayor y Promedio
10.- Suma de Impares
11.- Primer Elemento al Final
12.- Primer Elemento Par al Final
-----
0.- Salir
-----
Ingrese una alternativa: 3

LISTADO
-----
15      3      20      10      50

MENU DE OPCIONES
-----
1.- Crear el Arreglo
2.- Ingresar Datos
3.- Mostrar Datos
4.- Suma de Pares
5.- Eliminar un Elemento
6.- Modificar un Elemento
7.- Insertar un Elemento
```

En el arreglo 50, 15, 3, 20, 10 el primer número par es 50. La opción 12 lo mueve al final y el resultado pasa a ser 15, 3, 20, 10, 50, exactamente como exige el punto 8.

```
7.- Insertar un Elemento
8.- Sumar todos los Elementos
9.- Menor, Mayor y Promedio
10.- Suma de Impares
11.- Primer Elemento al Final
12.- Primer Elemento Par al Final
-----
0.- Salir
-----
Ingrese una alternativa: 0

SALIENDO DEL PROGRAMA
BUILD SUCCESSFUL (total time: 3 minutes 27 seconds)
```

Al ingresar la opción 0 se muestra “SALIENDO DEL PROGRAMA”. NetBeans finaliza la ejecución con BUILD SUCCESSFUL, lo que evidencia que el programa terminó sin errores de compilación o ejecución en la prueba realizada.

## **7. RESULTADOS OBTENIDOS**

La ejecución permitió comprobar que las ocho operaciones solicitadas funcionan sobre el arreglo y producen resultados coherentes con los datos ingresados. La eliminación redujo correctamente la cantidad lógica de elementos; la modificación sustituyó el valor indicado; la inserción amplió el arreglo y respetó la posición elegida; las operaciones numéricas devolvieron una suma total de 98, un menor de 3, un mayor de 50, un promedio de 19.6 y una suma de impares de 18; y los dos métodos de desplazamiento reorganizaron los elementos sin perder información.

La separación entre AplicaArreglo y Arreglo también permitió comprobar que la interacción con el usuario y la lógica de procesamiento pueden mantenerse independientes. Esta organización facilita identificar qué parte del programa captura opciones y cuál realiza las operaciones sobre los datos.

## **8. CONCLUSIONES**

La actividad permitió aplicar de manera práctica el uso de arreglos, ciclos for, estructuras condicionales, operadores aritméticos y métodos en Java. Cada requerimiento fue implementado como una operación independiente, lo que favorece la claridad del código y permite probar cada función por separado.

Las operaciones de eliminación, inserción y desplazamiento permitieron comprender que modificar la ubicación lógica de los datos requiere copiar o mover elementos dentro del arreglo. Asimismo, los cálculos de suma, menor, mayor y promedio reforzaron el uso de recorridos lineales y acumuladores.

Finalmente, la aplicación ejecutada en Apache NetBeans completó todas las pruebas previstas y terminó con BUILD SUCCESSFUL. El TDA y la organización en dos clases muestran una estructura básica de programación orientada a objetos adecuada para continuar desarrollando ejercicios de Algoritmos y Estructura de Datos.